Research Master's in Computer Science, Intelligent Pervasive Systems

Evaluating Coding Agent Quality

Application to Software Design Principles

Akram Ben Fekih

Supervised by Mrs. Maha Khemaja

ISSAT Sousse, University of Sousse · 2025/2026

Coding agents now submit real changes

- Agents such as **Claude Code**, **GitHub Copilot** and **OpenAI Codex** take a task and open a pull request.
- The AIDev dataset records **7.69 million** agent pull requests in 957k repositories.
- They are evaluated mainly on whether tests pass (HumanEval, SWE-bench).
- Passing tests says little about **design**: whether the next developer can change the code.

about half

of SWE-bench Verified patches that pass the tests would not be merged by maintainers (METR, 2026)

77%

of long-horizon agent trajectories show growing structural erosion (SlopCodeBench, 2026)

What is missing

Existing studies

- Benchmarks on functions or small sets of tasks
- LLMs as SOLID detectors on examples, not on agent output
- Smell counts on merged code only
- Metrics assumed to capture a principle, rarely validated

This thesis

- Real pull requests from three agents
- All five SOLID principles
- Merged, rejected *and* open requests
- Each detector's precision and recall **measured**
- Full replication in a second language

Research questions

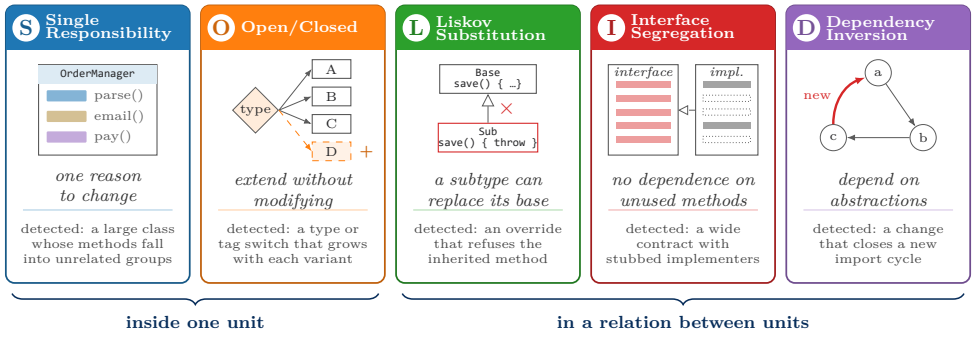
RQ1 **Agents and outcomes.** Do agents differ in SOLID adherence, and does merged code differ from code that was not merged?

RQ2 **Measurability.** Can SOLID be measured with deterministic detectors, and how well do they agree with a reading judge and with established analysers?

RQ3 **Locus of risk.** Do design problems lie inside single units or in relations between units?

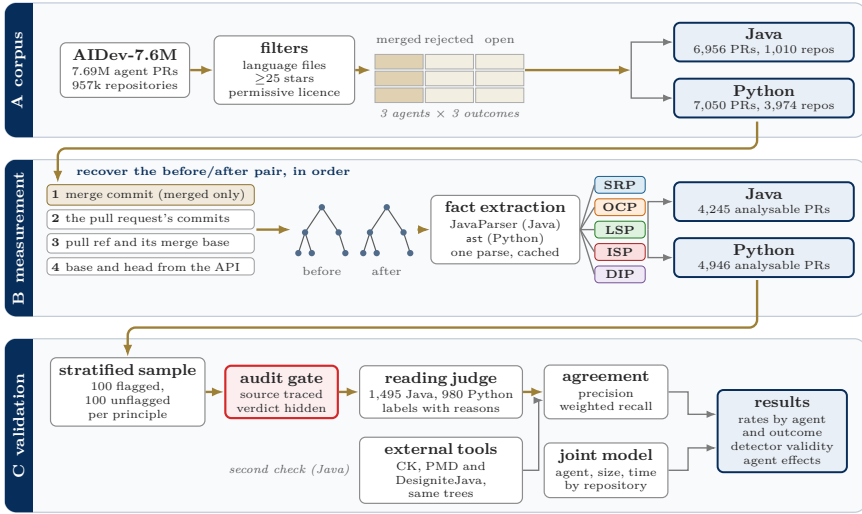
RQ4 **Generalisation.** Which answers hold again in Python?

Five principles, five detectors



Each detector looks for the structure a violation usually leaves in the code.

Method


A corpus
AIDev-7.6M
7.69M agent PRs
957k repositories
filters
language files
≥25 stars
permissive licence
merged rejected open
3 agents × 3 outcomes
Java
6,956 PRs, 1,010 repos
Python
7,050 PRs, 3,974 repos
B measurement
recover the before/after pair, in order
1 merge commit (merged only)
2 the pull request's commits
3 pull ref and its merge base
4 base and head from the API
before
after
fact extraction
JavaParser (Java)
ast (Python)
one parse, cached
SRP
OCP
LSP
ISP
DIP
Java
4,245 analysable PRs
Python
4,946 analysable PRs
C validation
stratified sample
100 flagged,
100 unflagged
per principle
audit gate
source traced
verdict hidden
reading judge
1,495 Java, 980 Python
labels with reasons
agreement
precision
weighted recall
results
rates by agent
and outcome
detector validity
agent effects
second check (Java)
external tools
CK, PMD and
DesigniteJava,
same trees
joint model
agent, size, time
by repository

Corpus

4,245
Java PRs
1,010 repositories

4,946
Python PRs
2,921 repositories

- Same three agents in both languages
- Balanced on agent $\times$ outcome
- $\geq$25 stars, permissive licence
- Python sample stratified by repository

Rejected is not refused

Only requests with a review record a judgement about the code.

Java: 171 of 1,177 unmerged requests were reviewed.
Python: 196 of 1,496.

Outcomes are therefore compared by stratum, and only where both sides were reconstructed the same way.

Charging only what the pull request changed



For a relation-level principle, a violation is charged when either side of the relation is in a touched file.

How the detectors were validated

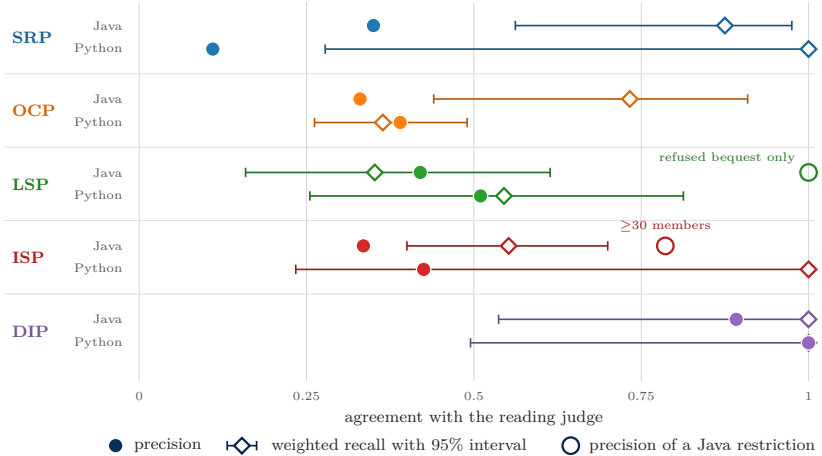
- 100 flagged + 100 unflagged entities per principle
- A reading judge (LLM) labels each one, **with a reason**, without seeing the verdict
- An audit gate: every label traces to the source shown
- Recall weighted by stratum size
- Holdout samples for the two refinements
- DIP (Java) and ISP (Python): every flag judged

2,475
entities judged
1,495 Java, 980 Python

89.7%
agreement when 174 Java entities were judged twice

0.966 → 0.352
Liskov recall, un-weighted vs weighted

RQ2: how faithful are the detectors?



Dependency Inversion is precise in both languages (0.892 Java, 1.000 Python). SRP, OCP and ISP are screens: most flags do not survive reading.

RQ2: a shape rule transfers, a tuned number does not

Refinement	Derivation sample	Java holdout	Python (unchanged)
LSP, refused bequest only	1.000 (31/31)	**1.000** (33/33)	**0.857** (6/7)
ISP, $\geq$30 members	0.862 (25/29)	0.704 (19/27)	0.200 (1/5)

- Refused bequest is **categorical**: a method body is only a throw, or it is not.
- The width of 30 was chosen where false positives stopped, and absorbed sample noise.
- In Python, 46 of the 51 confirmed Liskov violations rest on faults `javac` would reject: the compiler decides which faults can exist.

RQ2: agreement with published analysers (Java)

God Class detectors, 82,553 types	Cohen's κ
ours vs PMD `GodClass`	0.574
ours vs DesigniteJava	0.531
PMD vs DesigniteJava	0.384

Rank correlation with CK	ρ
weighted methods	0.977
lines of code	0.944
cohesion (LCOM1)	0.288

- 150 random repositories, default thresholds
- The rubric agrees with each tool more than the tools agree with each other
- 68.0% of our flags confirmed by at least one tool
- Cohesion metrics disagree everywhere, including within CK

RQ1: raw rates mostly measure change size

Mean Java files per pull request:

Claude Code **46.4** Copilot **18.5** Codex **10.9**

	Files	Claude	Copilot	Codex
OCP	1–3	1.3%	**1.9%**	0.4%
	11+	**31.3%**	19.3%	8.9%
DIP	1–3	0.4%	0.5%	**0.8%**
	11+	**22.3%**	13.7%	12.1%

- Claude Code has the highest pooled rate on all five principles.
- The ordering holds on large changes and breaks on small ones.
- So the agents must be compared **at the same change size**.

RQ1: the joint model

Per principle: agent + $\log$(files) + month, GEE clustered by repository.

	Java	Python
odds per doubling of files	1.88–1.95	2.01–2.18
Claude Code, OCP	2.22	2.09
95% interval	1.21–4.07	1.44–3.04
p	0.010	0.0001
time, per month	≈1	≈1

odds ratios against Copilot; time for the delta principles

- **Change size dominates** in both languages.
- **One agent difference replicates**: Claude Code on Open/Closed.
- It passes Bonferroni in Python; Java alone does not, so the claim rests on replication.
- Python's import-cycle effect (1.88, $p = 0.0096$) is not confirmed.

RQ1: merged against unmerged code

Java, tier-matched

Stratum	n	OCP	DIP
merged	1,521	8.3%	4.6%
unreviewed	1,006	7.5%	5.6%
reviewed, closed	135	12.6%	12.6%

Merged $\approx$ unreviewed.

Python, tier-matched

Stratum	n	OCP	DIP
merged	1,734	4.9%	3.1%
unreviewed	1,300	9.1%	4.5%
reviewed, closed	163	7.4%	8.6%

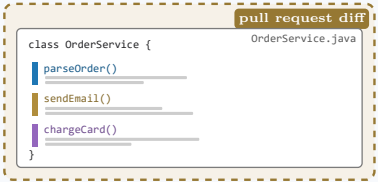
Merged is cleaner: unmerged odds 1.57 (OCP) and 2.43 (LSP) after size and agent.

The languages disagree. Most unmerged requests were never reviewed, so neither result shows what reviewers select on.

RQ3: where the problems live

Inside one unit

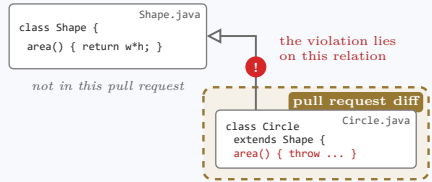
Single Responsibility, Open/Closed



The change can write the whole class, so one pull request can create the violation.

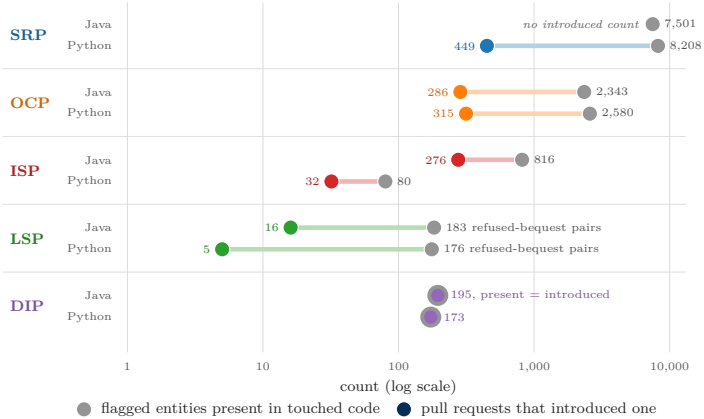
Between units

Liskov, Interface Segregation, Dependency Inversion



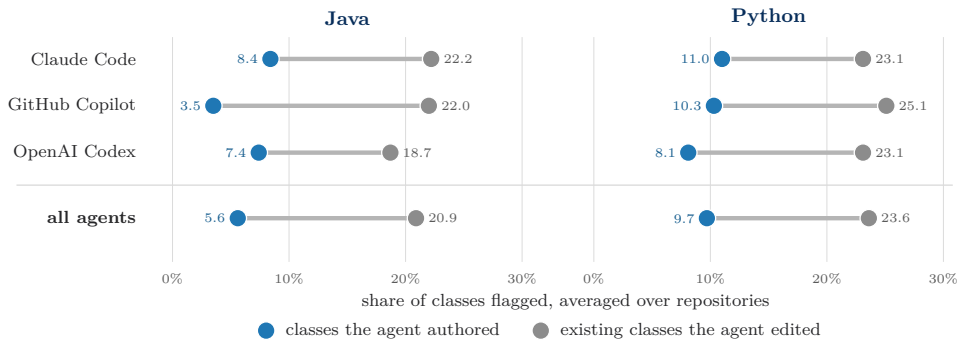
Each side is usually written by a different change, so these violations build up over many changes.

RQ3: most violations were already there



Refused bequest: 183 present vs 16 introducing PRs in Java, 176 vs 5 in Python. Between 0.4% and 4.1% of Java pull requests introduce a violation.

RQ3: agent-written classes are flagged less



Pooled: 7.7% vs 13.5% in Java, 8.9% vs 12.1% in Python. The groups differ in age and size, so this describes the corpus; it does not rank agents against people.

RQ4: what generalises

Result	Java	Python
Change size dominates	OR 1.88–1.95	**holds** 2.01–2.18
Claude Code on Open/Closed	2.22 (1.21–4.07)	**holds** 2.09 (1.44–3.04)
Present far exceeds introduced	183 vs 16	**holds** 176 vs 5
Authored flagged less than edited	7.7% vs 13.5%	**holds** 8.9% vs 12.1%
Shape rule transfers, number does not	1.000 vs 0.704	**holds** 0.857 vs 0.200
Merged equals unreviewed	yes	**does not hold**
SRP screen precision	0.350	**drops** to 0.110

Contributions and practical use

Contributions

- A validated SOLID instrument over real agent pull requests
- A corpus with merged, rejected and open requests
- A map of which principles a rule measures and which it only screens
- Evidence in two languages that most problems predate the agent's change
- A replication with the same agents

For a review check

- Add **refused bequest**: 64/64 in Java, 6/7 in Python
- In dynamic languages, also check that overrides accept the base's calls
- Do not rely on interface-width thresholds
- Treat SRP and OCP flags as prompts for discussion

Limitations

- SOLID adherence is not measured design quality or cost
- Labels come from an LLM judge of the same family as one agent
- No independent-tool check for OCP, ISP or Python
- Agents chose their tasks: associations, not causes
- Few reviewed rejections (171 Java, 196 Python)
- One snapshot per pull request, no long-term erosion
- Both corpora from one dataset; 7.0% of Python source lost
- Small counts for relation-level principles

Conclusion and future work

Most design problems in the code these agents touch were there before them; how much an agent changes matters more than which agent it is. Both results hold in Java and in Python.

Next steps

- Recognise closed sets from external specifications (OCP)
- Compare OCP with DesigniteJava's type-conditional smells
- Re-judge Python samples; a second judge
- A corpus with many reviewed rejections
- Repository history as the unit of analysis
- A third language with structural typing

Thank you

Questions